

Implementação do GRASP para Otimização do Índice de Qualidade de Caminhabilidade (IQC)

Relatório técnico detalhado: teoria e implementação passo a passo

Anderson Oliveira

15 de junho de 2026

Resumo

Este relatório documenta a implementação do *Greedy Randomized Adaptive Search Procedure* (GRASP), na sua forma clássica de duas fases descrita por Resende e Ribeiro, aplicado ao problema de alocação de Pontos de Interesse (POIs) que maximiza o Índice de Qualidade de Caminhabilidade (IQC) total de uma malha de hexágonos H3. A primeira parte apresenta a teoria do GRASP utilizada (multi-start, construção gulosa-aleatória com Lista Restrita de Candidatos e busca local). A segunda parte descreve, função a função, como o método foi implementado, incluindo as estruturas de dados em `ndarray`, a função objetivo (pesos CRITIC + IQC) e as decisões de projeto. Por fim, uma instância didática é executada e *ilustrada passo a passo*, mostrando como a matriz de alocação evolui, como o impacto se propaga entre hexágonos e como o IQC é recalculado a cada iteração.

Sumário

1	Introdução e contexto	2
2	Formalização do problema	2
2.1	Conjunto base e variável de decisão	2
2.2	Restrição de viabilidade	2
2.3	Propagação espaço-temporal do impacto	3
2.4	Função objetivo: CRITIC + IQC	3
3	Teoria do GRASP utilizada	4
3.1	Visão geral (multi-start)	4
3.2	Fase de construção e a Lista Restrita de Candidatos (RCL)	4
3.3	Fase de busca local	5
4	Implementação	6
4.1	Onde o método se encaixa	6
4.2	Representação numérica e propagação	6

4.3	Função de avaliação <code>_evaluate</code>	6
4.4	Fase de construção <code>_greedy_randomized_construction</code>	6
4.5	Fase de busca local <code>_local_search</code>	7
4.6	Multi-start e retorno <code>run_grasp</code>	8
4.7	Paralelismo e cache	9
4.8	Parâmetros e decisões de projeto	9
5	Execução ilustrada passo a passo	10
5.1	Instância	10
5.2	Construção — passo a passo	10
5.3	Propagação do impacto	12
5.4	Recálculo do CRITIC e do IQC	12
5.5	Resultado e leitura	12
6	Arquivos gerados para análise estatística	13
6.1	Organização: duas camadas	13
6.2	Identificadores comuns (chaves de junção)	14
6.3	<code>all_runs_summary.csv</code> / <code>run_summary.csv</code>	14
6.4	<code>all_per_seed.csv</code> / <code>per_seed.csv</code>	16
6.5	<code>all_per_hexagon_iqc.csv</code> / <code>per_hexagon_iqc.csv</code>	16
6.6	<code>per_hexagon_indicators.csv</code>	17
6.7	<code>all_allocations.csv</code> / <code>allocation.csv</code>	17
6.8	<code>critic_weights.csv</code>	18
6.9	<code>convergence.csv</code>	18
6.10	<code>run_metadata.json</code>	18
6.11	<code>run_report.txt</code>	18
6.12	Receituário de análises	18
7	Considerações finais e extensões	19

1 Introdução e contexto

O projeto calcula, para cada hexágono H3 de uma localidade, um conjunto de indicadores de caminhabilidade (saúde, educação, transporte, segurança, etc.) e deles deriva o **IQC** (Índice de Qualidade de Caminhabilidade) via pesos **CRITIC**. A etapa de otimização busca responder a uma pergunta de planejamento urbano:

Dado um orçamento de B novos POIs, em quais hexágonos e de quais dimensões devemos inseri-los para maximizar o IQC total da região?

Trata-se de um problema combinatório: o número de maneiras de distribuir B POIs entre os pares (hexágono-fonte, dimensão) cresce explosivamente. Métodos exatos são inviáveis para instâncias reais, o que motiva o uso de uma **meta-heurística**. Escolhemos o GRASP por três razões alinhadas ao texto de Resende e Ribeiro: (i) requer poucos parâmetros (essencialmente o parâmetro α da lista de candidatos); (ii) tem estrutura simples de implementar sobre blocos já disponíveis (uma função de construção e uma de busca local); e (iii) é naturalmente multi-start, o que o torna robusto à aleatoriedade das sementes.

2 Formalização do problema

2.1 Conjunto base e variável de decisão

Seja $H = \{h_1, \dots, h_n\}$ o conjunto dos n hexágonos e $D = \{d_1, \dots, d_m\}$ o conjunto das m dimensões de POI alocáveis (saúde, transporte, etc.). O **conjunto base** (*ground set*) do GRASP é

$$E = H_{\text{fonte}} \times D, \quad (1)$$

onde $H_{\text{fonte}} \subseteq H$ são os hexágonos elegíveis para receber POIs. Cada *elemento* $e = (h, d) \in E$ representa “inserir uma unidade de POI da dimensão d no hexágono-fonte h ”.

A solução é uma **matriz de alocação** (chamada no código de `candidate_matrix`)

$$X \in \mathbb{Z}_{\geq 0}^{n \times m}, \quad X_{h,d} = \text{número de POIs da dimensão } d \text{ alocados no hexágono } h. \quad (2)$$

2.2 Restrição de viabilidade

A única restrição é o orçamento total:

$$\sum_{h=1}^n \sum_{d=1}^m X_{h,d} = B. \quad (3)$$

Como a construção do GRASP insere exatamente um POI por passo e para em B passos, e a busca local apenas *move* POIs (preservando a soma), a restrição (3) é satisfeita por construção — nenhum procedimento de reparo é necessário.

2.3 Propagação espaço-temporal do impacto

Inserir um POI num hexágono-fonte s não beneficia apenas s : o efeito se *propaga* aos hexágonos-alvo t alcançáveis a pé, ponderado por um fator de decaimento temporal $\alpha_{s,t} \in (0, 1]$ (cosseno do tempo de caminhada, campo `alpha_20`). Seja P o conjunto de pares fonte-alvo com $\alpha_{s,t} > 0$. A matriz final de indicadores é

$$M = M^0 + \Delta, \quad \Delta_{t,j(d)} = \sum_{(s,t) \in P} \alpha_{s,t} X_{s,d}, \quad (4)$$

onde M^0 é a matriz *baseline* de indicadores ($n \times 11$), $j(d)$ é a coluna do indicador correspondente à dimensão d , e a soma acumula a contribuição de todos os POIs que, a partir de suas fontes, alcançam o alvo t .

2.4 Função objetivo: CRITIC + IQC

Dada a matriz final M , a função objetivo executa três etapas (implementadas em `objective_function`):

(1) **Pesos CRITIC.** Normaliza-se cada coluna por *min-max*,

$$\hat{M}_{ij} = \frac{M_{ij} - \min_i M_{ij}}{\max_i M_{ij} - \min_i M_{ij}}, \quad (5)$$

calcula-se o desvio-padrão σ_j de cada coluna normalizada e o *conflito* $c_j = \sum_k (1 - |\rho_{jk}|)$, onde ρ_{jk} é a correlação de Pearson entre os indicadores j e k . O peso de cada indicador é

$$w_j = \frac{\sigma_j c_j}{\sum_l \sigma_l c_l}. \quad (6)$$

(2) **IQC por hexágono.** É a média ponderada dos indicadores normalizados:

$$\text{IQC}_i = \sum_j w_j \hat{M}_{ij}. \quad (7)$$

(3) **Objetivo global.** Soma do IQC sobre todos os hexágonos, a ser **maximizada**:

$$\boxed{f(X) = \sum_{i=1}^n \text{IQC}_i} \quad (\text{direção: maximizar}). \quad (8)$$

Observação crucial. Os pesos CRITIC (6) e a normalização (7) dependem de *todas* as colunas e linhas de M . Logo, f é **não-separável**: o ganho de inserir um POI não é constante — muda conforme a solução parcial atual. Isso encaixa exatamente no caráter *adaptativo* do GRASP (a função gulosa é reavaliada a cada passo), mas torna cada avaliação cara, o que orienta as decisões de eficiência da Seção 4.

3 Teoria do GRASP utilizada

O GRASP (Feo e Resende, 1995; Resende e Ribeiro, 2016, 2019) é uma meta-heurística *multi-start*: cada iteração é independente e consiste em duas fases — **construção** gulosa-aleatória e **busca local** — e a melhor solução global é mantida ao final.

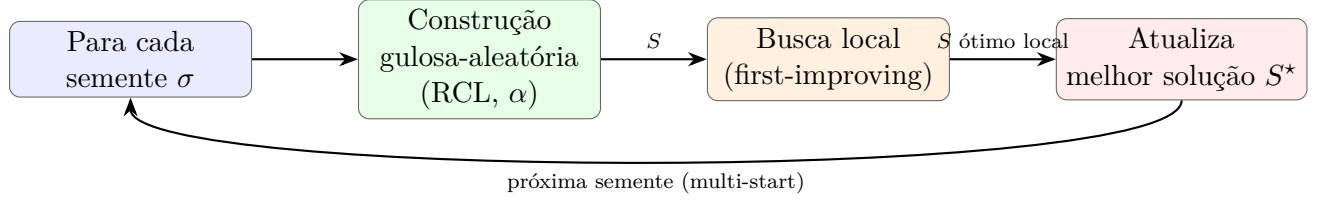


Figura 1: Estrutura geral do GRASP: laço multi-start sobre as sementes; cada iteração constrói uma solução, refina-a por busca local e atualiza a melhor solução encontrada.

3.1 Visão geral (multi-start)

O Algoritmo 1 resume o laço principal, adaptado para **maximização** (o texto original trata minimização). `Max_Iterations` corresponde, na nossa implementação, ao número de sementes disponíveis.

Algorithm 1 GRASP (versão de maximização)

```

1:  $f^* \leftarrow -\infty$ ;  $S^* \leftarrow \emptyset$ 
2: for all semente  $\sigma$  em seeds do
3:    $S \leftarrow \text{CONSTRUCAOGULOSA-ALEATORIA}(\alpha, \sigma)$ 
4:    $S \leftarrow \text{BUSCALOCAL}(S)$ 
5:   if  $f(S) > f^*$  then
6:      $S^* \leftarrow S$ ;  $f^* \leftarrow f(S)$ 
7:   end if
8: end for
9: return  $S^*$ 
  
```

3.2 Fase de construção e a Lista Restrita de Candidatos (RCL)

A construção monta a solução *um elemento por vez*. A cada passo avalia-se, para cada elemento candidato e , o **ganho incremental** $g(e)$ que sua inserção provoca na função objetivo. Constrói-se então a **Lista Restrita de Candidatos** (RCL) com os *melhores* elementos e sorteia-se um deles ao acaso. Três aspectos coexistem:

- **Guloso** — só entram na RCL os elementos de maior ganho;
- **Probabilístico** — escolhe-se aleatoriamente *dentro* da RCL;
- **Adaptativo** — após cada inserção os ganhos são reavaliados.

A RCL é controlada pelo limiar $\alpha \in [0, 1]$. Para **maximização**, sendo $g_{\max}$ e $g_{\min}$ o maior e o menor ganho dos candidatos, a lista é

$$\text{RCL} = \left\{ e : g(e) \geq g_{\max} - \alpha (g_{\max} - g_{\min}) \right\}. \quad (9)$$

Os extremos têm interpretação direta: $\alpha = 0$ produz um algoritmo *puramente guloso* (só o melhor elemento entra na RCL), enquanto $\alpha = 1$ produz uma *construção aleatória* (todos entram). Resende e Ribeiro mostram empiricamente que valores intermediários ($\alpha \approx 0,2$) oferecem o melhor compromisso entre média e variância das soluções; adotamos esse valor.

Algorithm 2 Construção gulosa-aleatória (maximização, *sampled greedy*)

```

1:  $S \leftarrow$  matriz nula  $n \times m$ ;  $v \leftarrow f(S)$ 
2: for  $k = 1, \dots, B$  do
3:    $\mathcal{C} \leftarrow$  amostra aleatória de  $p$  pares  $(h, d)$  do conjunto base  $E$ 
4:   for all  $(h, d) \in \mathcal{C}$  do
5:      $g(h, d) \leftarrow f(S \text{ com } X_{h,d}+1) - v$  ▷ ganho incremental
6:   end for
7:    $g_{\max} \leftarrow \max g$ ;  $g_{\min} \leftarrow \min g$ 
8:    $\text{RCL} \leftarrow \{(h, d) \in \mathcal{C} : g(h, d) \geq g_{\max} - \alpha(g_{\max} - g_{\min})\}$ 
9:   sorteie  $(h^*, d^*)$  uniformemente da RCL
10:   $X_{h^*, d^*} += 1$ ;  $v \leftarrow v + g(h^*, d^*)$ 
11: end for
12: return  $S$ 

```

3.3 Fase de busca local

A solução construída raramente é ótima localmente. A busca local substitui iterativamente a solução corrente por um vizinho melhor, parando no primeiro ótimo local. Como o orçamento é fixo, a vizinhança natural é o **movimento de um POI**: retirar uma unidade de um par (h, d) ocupado e inseri-la em outro par (h', d') . Usa-se a estratégia *first-improving* (aceita a primeira melhoria encontrada), recomendada por Resende e Ribeiro por ser mais rápida e menos sujeita à convergência prematura que a *best-improving*.

Algorithm 3 Busca local *first-improving* (movimento de 1 POI)

```

1:  $v \leftarrow f(S)$ 
2: repeat
3:    $\text{melhorou} \leftarrow \text{falso}$ 
4:   for all  $(h, d)$  com  $X_{h,d} > 0$  (ordem aleatória) do
5:     for all  $(h', d') \neq (h, d)$  (ordem aleatória) do
6:        $S' \leftarrow S$  movendo 1 POI de  $(h, d)$  para  $(h', d')$ 
7:       if  $f(S') > v + \epsilon$  then
8:          $S \leftarrow S'$ ;  $v \leftarrow f(S')$ ;  $\text{melhorou} \leftarrow \text{verdadeiro}$ ; break
9:       end if
10:    end for
11:    if  $\text{melhorou}$  then break
12:  end if
13: end for
14: until  $\neg \text{melhorou}$  ou orçamento de avaliações esgotado
15: return  $S$ 

```

4 Implementação

A implementação reside em `metaheuristics/methods/grasp.py` e expõe a função `run_grasp(context)`, despachada automaticamente pelo otimizador quando o usuário escolhe a opção **B**. Esta seção descreve cada parte.

4.1 Onde o método se encaixa

O orquestrador `walk_meta_opt` prepara um objeto `context` (pacote de dados) e o entrega ao `runner` do método. Os campos relevantes para o GRASP são:

- `context.objective_state_nd` — estado numérico pré-compilado (matriz baseline, índices fonte/alvo, valores α , mapeamentos);
- `context.budget` — orçamento B de POIs;
- `context.seeds` — lista de sementes (define o multi-start);
- `context.source_hex_ids` e `context.dimensions` — formam o conjunto base E .

4.2 Representação numérica e propagação

Toda a avaliação opera em `ndarray`, sem `pandas` no laço quente. A solução é a `candidate_matrix` X de forma (n, m) ; o estado pré-compilado `objective_state_nd` guarda `baseline_matrix` (M^0), os arrays `source_indices`, `target_indices`, `alpha_values` (que representam o conjunto P e os $\alpha_{s,t}$) e o mapa `candidate_to_indicator_indices` (que liga cada coluna de X à coluna correspondente em M). A propagação (4) é uma soma dispersa (*scatter-add*) vetorizada.

4.3 Função de avaliação `_evaluate`

Encapsula a Eq. (4) seguida da Eq. (8): dada X , propaga o impacto e devolve $\sum_i \text{IQC}_i$.

```
1 def _evaluate(candidate_matrix, objective_state):
2     """Score a candidate matrix: propagate impacts then return sum(IQC)."""
3     final_indicator_matrix = build_final_indicator_matrix_nd(
4         candidate_matrix=candidate_matrix,
5         objective_state=objective_state,
6     )
7     return objective_function(final_indicator_matrix=final_indicator_matrix)['objective_value']
```

Listing 1: Avaliação de um candidato: propaga o impacto e retorna a soma do IQC.

4.4 Fase de construção `_greedy_randomized_construction`

Implementa o Algoritmo 2. Pontos a destacar:

- começa de $X = \mathbf{0}$ e insere 1 POI por passo, até B ;
- por passo, amostra `sample_size` pares (*sampled greedy*) e mede o ganho de cada um chamando `_evaluate` com o POI somado e depois revertido;

- monta a RCL pela Eq. (9) e sorteia um índice dela;
- a atualização `current_value += gains[chosen]` é *exata*, pois `gains[chosen]` já é a diferença entre o valor avaliado e o valor corrente.

```

1 def _greedy_randomized_construction(objective_state, elements, budget,
2                                     alpha, sample_size, rng):
3     n_hex = len(objective_state.h3_ids)
4     n_dims = len(objective_state.candidate_dimensions)
5     candidate = np.zeros((n_hex, n_dims), dtype=np.float64)
6     current_value = _evaluate(candidate, objective_state)
7
8     for _ in range(budget):
9         if sample_size < len(elements):
10             sampled = rng.sample(elements, sample_size)
11         else:
12             sampled = list(elements)
13
14         gains = np.empty(len(sampled), dtype=np.float64)
15         for i, (row, col) in enumerate(sampled):
16             candidate[row, col] += 1.0
17             gains[i] = _evaluate(candidate, objective_state) - current_value
18             candidate[row, col] -= 1.0
19
20         g_max = float(gains.max())
21         g_min = float(gains.min())
22         threshold = g_max - alpha * (g_max - g_min)          # RCL de maximizacao
23         rcl = [i for i in range(len(sampled)) if gains[i] >= threshold]
24
25         chosen = rng.choice(rcl)
26         row, col = sampled[chosen]
27         candidate[row, col] += 1.0
28         current_value += float(gains[chosen])
29
30     return candidate, current_value

```

Listing 2: Construção gulosa-aleatória com RCL amostrada.

4.5 Fase de busca local `_local_search`

Implementa o Algoritmo 3 com *first-improving*. A vizinhança é o movimento de 1 POI; a varredura é aleatorizada (ordem dos pares ocupados e dos destinos) e limitada por um orçamento de avaliações `max_evals`, que garante tempo de execução previsível mesmo quando a função objetivo é cara.

```

1 def _local_search(candidate, objective_state, elements, rng, max_evals):
2     current_value = _evaluate(candidate, objective_state)
3     evals = 0
4     improved = True
5
6     while improved and evals < max_evals:
7         improved = False

```



```

8      occupied = [(int(r), int(c)) for r, c in zip(*np.nonzero(candidate))]
9      rng.shuffle(occupied)
10
11     for (row, col) in occupied:
12         destinations = list(elements)
13         rng.shuffle(destinations)
14
15         for (row2, col2) in destinations:
16             if row2 == row and col2 == col:
17                 continue
18             if evals >= max_evals:
19                 break
20             candidate[row, col] -= 1.0
21             candidate[row2, col2] += 1.0
22             value = _evaluate(candidate, objective_state)
23             evals += 1
24             if value > current_value + IMPROVEMENT_EPS:
25                 current_value = value
26                 improved = True
27                 break                                     # first-improving: aceita e
reinicia
28                 candidate[row, col] += 1.0               # reverte
29                 candidate[row2, col2] -= 1.0
30
31             if improved or evals >= max_evals:
32                 break
33
34     return candidate, current_value

```

Listing 3: Busca local first-improving por movimento de POI.

4.6 Multi-start e retorno run_grasp

O laço principal (Algoritmo 1) percorre as sementes, executa construção + busca local por semente e guarda a melhor solução. Antes do laço, calcula o objetivo *baseline* (matriz $X = \mathbf{0}$) para medir o ganho. O conjunto base é restrito aos hexágonos-fonte que de fato propagam impacto (`np.unique(source_indices)`), evitando avaliar pares inócuos.

```

1 elements = [(int(row), col) for row in source_row_indices for col in range(
    n_dims)]
2 seeds = context.seeds if context.seeds else [0]
3 baseline_value = _evaluate(np.zeros((len(objective_state.h3_ids), n_dims)),
    objective_state)
4
5 best_value, best_candidate, best_seed = float('-inf'), None, None
6 for seed in seeds:
7     rng = random.Random(int(seed))
8     candidate, _ = _greedy_randomized_construction(
9         objective_state, elements, context.budget, RCL_ALPHA,
10         CONSTRUCTION_SAMPLE_SIZE, rng)
11     candidate, value = _local_search(
12         candidate, objective_state, elements, rng, LOCAL_SEARCH_MAX_EVALS)

```

```

13     if value > best_value:
14         best_value, best_candidate, best_seed = value, candidate.copy(), int(
            seed)

```

Listing 4: Laço multi-start e montagem do resultado (trecho central).

O dicionário de retorno traz `best_objective_value` e um `best_solution_summary` com a semente vencedora, o `baseline_objective`, o `improvement` e a `allocation` final — a lista de itens `{h3_id, dimension, quantity}` obtida convertendo a matriz X de volta para forma legível (`_candidate_matrix_to_items`).

4.7 Paralelismo e cache

O laço multi-start é *embaraçosamente paralelo*: cada semente executa uma construção e uma busca local independentes, sem estado compartilhado. Por isso despachamos **uma semente por processo**, distribuídas em $\max(\text{núcleos} - 1, 1)$ *workers* (via `ProcessPoolExecutor`). Essa é a estratégia de paralelização independente do GRASP, que Resende e Ribeiro reportam levar a *speedups* próximos do linear.

Três cuidados garantem corretude e robustez:

- **Determinismo.** Cada semente possui o seu próprio gerador `random.Random(seed)`, de modo que o resultado independe do número de *workers* e da ordem de conclusão. Os resultados são reordenados por semente antes de montar as estatísticas, então os arquivos de saída são idênticos em execução serial ou paralela (verificado por testes de igualdade serial = paralelo).
- **Sem *oversubscription*.** Como cada *worker* já é um processo, fixamos o BLAS do `numpy` em uma *thread* por *worker* (variáveis `OMP_NUM_THREADS` etc.), evitando contenção de CPU.
- **Tolerância a falhas.** Se o ambiente não permitir a criação de processos, o método captura a exceção e executa *serialmente*, sem interromper a otimização.

Adicionalmente, cada *worker* mantém um **cache** em memória (por semente) que memoiza a avaliação pela *conteúdo* da matriz candidata (`candidate.tobytes()`), evitando recomputar matrizes idênticas. Como a função objetivo é não-separável (CRITIC global), matrizes candidatas raramente se repetem dentro da construção; o cache, portanto, ajuda sobretudo na busca local e *nunca* prejudica (apenas memoiza), tendo um teto de tamanho para não esgotar a memória. O ganho de desempenho principal vem do paralelismo.

4.8 Parâmetros e decisões de projeto

As escolhas adotadas e seus controles (constantes no topo do módulo) estão na Tabela 1. As três primeiras refletem as decisões feitas: α fixo, busca local *first-improving* e *sampled greedy*. Os dois freios de custo (`CONSTRUCTION_SAMPLE_SIZE` e `LOCAL_SEARCH_MAX_EVALS`) existem porque a função objetivo recalcula o CRITIC globalmente a cada avaliação; as quatro últimas controlam o paralelismo e o cache (Seção 4.7).

Tabela 1: Parâmetros da implementação.

Constante	Valor	Papel
RCL_ALPHA	0,2	limiar da RCL (Eq. 9)
CONSTRUCTION_SAMPLE_SIZE	64	pares avaliados por passo (<i>sampled greedy</i>)
LOCAL_SEARCH_MAX_EVALS	4000	teto de avaliações da busca local por semente
IMPROVEMENT_EPS	10^{-9}	ganho mínimo para aceitar melhoria
GRASP_PARALLEL	True	liga o multi-start paralelo
GRASP_MAX_WORKERS	None	None = núcleos-1; ou inteiro fixo
GRASP_EVAL_CACHE	True	memoização da avaliação por <i>worker</i>
GRASP_EVAL_CACHE_MAXSIZE	5×10^5	teto de entradas do cache por semente

5 Execução ilustrada passo a passo

Para tornar a mecânica concreta, executamos o GRASP numa **instância didática** reduzida (números reais produzidos pelo próprio código). Para facilitar a visualização usamos 3 indicadores e 2 dimensões alocáveis; a implementação real usa 11 indicadores e 7 dimensões, mas a lógica é idêntica.

5.1 Instância

Hexágonos: A, B, C, D. **Indicadores:** S_saude, T_transporte, I_seguranca.

Dimensões alocáveis: S_saude (coluna 0) e T_transporte (coluna 1). **Orçamento:** $B = 3$.

Propagação: cada fonte s impacta a si mesma ($\alpha = 1,0$) e o vizinho seguinte ($\alpha = 0,5$), no ciclo $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$.

Tabela 2: Matriz *baseline* de indicadores M^0 . Objetivo baseline $f(\mathbf{0}) = 1,8389$.

Hex	S_saude	T_transporte	I_seguranca
A	2	5	3
B	4	1	6
C	1	3	2
D	5	2	4

5.2 Construção — passo a passo

A cada passo mostramos o ganho de *todos* os 8 elementos (aqui `sample_size` cobre o conjunto base inteiro, então não há amostragem), a RCL (**azul**) e o elemento sorteado (**laranja**). À direita, a matriz de alocação X resultante, com a célula alterada em **verde**.

Passo 1. $g_{\max} = 0,1400$, $g_{\min} = -0,1572$, $\text{limiar} = 0,1400 - 0,2 \cdot (0,1400 + 0,1572) = 0,0806$.

Elemento (h, d)	ganho g
(A, S_saude)	+0,1400
(A, T_transporte)	-0,1297
(B, S_saude)	+0,0378
(B, T_transporte)	-0,1061
(C, S_saude)	-0,1572
(C, T_transporte)	+0,1287
(D, S_saude)	-0,0325
(D, T_transporte)	+0,0569

X após o passo 1:		
	S_saude	T_transp.
A	1	0
B	0	0
C	0	0
D	0	0
$f = 1,9789$		

A RCL contém os dois elementos acima do limiar; o sorteio escolheu (A, S_saude).

Passo 2. $g_{\max} = 0,1714$, $g_{\min} = -0,1531$, limiar = 0,1065. Note como os ganhos *mudaram* em relação ao passo 1 — é o caráter adaptativo: a solução parcial alterou o CRITIC e a normalização.

Elemento (h, d)	ganho g
(A, S_saude)	+0,1714
(A, T_transporte)	-0,1531
(B, S_saude)	-0,0723
(B, T_transporte)	-0,1355
(C, S_saude)	-0,1385
(C, T_transporte)	+0,1626
(D, S_saude)	-0,0583
(D, T_transporte)	+0,0661

X após o passo 2:		
	S_saude	T_transp.
A	2	0
B	0	0
C	0	0
D	0	0
$f = 2,1503$		

Sorteio escolheu novamente (A, S_saude): agora $X_{A, S_saude} = 2$.

Passo 3. $g_{\max} = 0,1786$, $g_{\min} = -0,2037$, limiar = 0,1021. O ganho de mais um POI de saúde em A caiu para +0,0269 (rendimentos decrescentes), e agora só (C, T_transporte) supera o limiar.

Elemento (h, d)	ganho g
(A, S_saude)	+0,0269
(A, T_transporte)	-0,1654
(B, S_saude)	-0,2037
(B, T_transporte)	-0,1495
(C, S_saude)	-0,1446
(C, T_transporte)	+0,1786
(D, S_saude)	-0,1017
(D, T_transporte)	+0,0658

X após o passo 3 (final):

	S_saude	T_transp.
A	2	0
B	0	0
C	0	1
D	0	0

$$f = 2,3289$$

5.3 Propagação do impacto

A solução construída aloca 2 POIs de S_saude na fonte A e 1 POI de T_transporte na fonte C. Aplicando a Eq. (4):

- **A (2 POIs de saúde):** $A \rightarrow A$ com $\alpha = 1,0$ soma +2,0 à saúde de A; $A \rightarrow B$ com $\alpha = 0,5$ soma +1,0 à saúde de B.
- **C (1 POI de transporte):** $C \rightarrow C$ com $\alpha = 1,0$ soma +1,0 ao transporte de C; $C \rightarrow D$ com $\alpha = 0,5$ soma +0,5 ao transporte de D.

Tabela 3: Matriz final $M = M^0 + \Delta$. Em verde, as células alteradas pela propagação.

Hex	S_saude	T_transporte	I_seguranca
A	4 (2+2)	5	3
B	5 (4+1)	1	6
C	1	4 (3+1)	2
D	5	2,5 (2+0,5)	4

5.4 Recálculo do CRITIC e do IQC

Sobre a matriz final, a função objetivo recalcula os pesos CRITIC (6) e o IQC (7):

$$w = (\underbrace{0,4436}_{S_saude}, \underbrace{0,3608}_{T_transporte}, \underbrace{0,1956}_{I_seguranca}).$$

5.5 Resultado e leitura

Partindo do baseline $f(0) = 1,8389$, a fase de construção já elevou o objetivo para $f(X) = 2,3289$, um **ganho de** +0,4900. A busca local parte daqui tentando movimentos de 1 POI; nesta instância mínima a solução construída já é ótima local. Em instâncias reais, é a combinação *construção (diversificação via RCL) + busca local (intensificação) + multi-start* que produz soluções de alta qualidade, exatamente como descrito por Resende e Ribeiro.

Tabela 4: IQC por hexágono após a construção e objetivo global.

Hex	IQC _i
A	0,7424
B	0,6392
C	0,2706
D	0,6767
$f(X) = \sum_i \text{IQC}_i$	2,3289

A leitura urbanística do resultado é direta: concentrar POIs de saúde no hexágono A (que também irriga B) e um POI de transporte em C (que alcança D) foi a alocação de melhor retorno de IQC sob o orçamento dado.

6 Arquivos gerados para análise estatística

Toda execução do otimizador grava automaticamente um conjunto de arquivos projetados para análise estatística reprodutível e publicação. Esta seção descreve cada arquivo, o que ele representa, como pode ser usado, quais métodos estatísticos são adequados e o significado de *cada* atributo (coluna).

6.1 Organização: duas camadas

Os arquivos ficam em `data/optimization_runs/` e se organizam em duas camadas complementares (Figura 2):

1. **Arquivos-mestre acumulativos** (na raiz): cada execução *acrescenta* linhas. São os arquivos lidos para análises *entre* execuções (comparar métodos, perfis, localidades).
2. **Pasta por execução** (`<key_location>/<perfil>/<método>/<timestamp>__<id>/`): guarda o detalhe completo e reprodutível de uma única execução.

```
data/optimization_runs/
|-- all_runs_summary.csv          (mestre: 1 linha por execucao)
|-- all_per_seed.csv             (mestre: 1 linha por execucao x semente)
|-- all_per_hexagon_iqc.csv      (mestre: 1 linha por execucao x hexagono)
|-- all_allocations.csv         (mestre: 1 linha por execucao x POI alocado)
'-- <key_location>/<perfil>/<metodo>/<timestamp>__<id>/
    |-- run_metadata.json        metadados completos (reprodutibilidade)
    |-- run_report.txt           resumo legivel
    |-- run_summary.csv          resumo de 1 linha (esta execucao)
    |-- per_seed.csv             resultado por semente
    |-- per_hexagon_iqc.csv      IQC baseline vs otimizado por hexagono
    |-- per_hexagon_indicators.csv indicadores (formato longo)
    |-- allocation.csv           melhor alocao
    |-- critic_weights.csv       pesos CRITIC baseline vs otimizado
    |-- convergence.csv         objetivo por passo de construcao
```

Figura 2: Estrutura de diretórios dos arquivos de saída.

6.2 Identificadores comuns (chaves de junção)

As quatro tabelas-mestre compartilham um conjunto de colunas de identificação, que servem de *chaves* para cruzamentos (`merge/join`) e agrupamentos (`group_by`). Elas aparecem em todos os arquivos longos e não são repetidas nas tabelas de atributos seguintes.

Atributo	Tipo	Descrição
<code>run_id</code>	texto	Identificador único da execução (UUID curto). Chave primária que liga <i>todos</i> os arquivos de uma mesma execução.
<code>location</code>	texto	Nome da localidade (legível).
<code>key_location</code>	texto	Chave saneada da localidade (usada em caminhos).
<code>walking_profile</code>	texto	Perfil de caminhada (<code>average_adult</code> , <code>elderly</code> , <code>athlete</code>).
<code>method_code</code>	texto	Código do método (B = GRASP).

6.3 `all_runs_summary.csv` / `run_summary.csv`

O que representa. Uma linha por execução, com todos os indicadores agregados. É o arquivo *central* para estatística: cada linha é uma observação experimental (uma combinação localidade × perfil × método × conjunto de sementes).

Como usar e quais métodos aplicar.

- *Comparar métodos/perfis/localidades:* agrupar por fator e comparar `best_objective` ou `improvement_pct`. Testar normalidade (Shapiro–Wilk) e homocedasticidade (Levene); se atendidas, **ANOVA fatorial** com pós-teste de **Tukey HSD**; caso contrário, **Kruskal–Wallis** (ou **Friedman**, se pareado pela mesma instância localidade×perfil) com pós-teste de **Dunn/Nemenyi**. Reportar tamanho de efeito (η^2 , ϵ^2 ou δ de Cliff).
- *Modelar o ganho:* regressão linear (ou modelo misto) `improvement_pct` ~ método+perfil+localidade + budget.
- *Robustez do método:* usar `obj_seed_std` / `obj_seed_mean` (coeficiente de variação) como medida de estabilidade entre sementes.

Atributos — identificação e estado.

Atributo	Tipo	Descrição
<code>run_id</code>	texto	Identificador único da execução.
<code>timestamp_utc</code>	ISO-8601	Data/hora de início em UTC.
<code>location</code> , <code>key_location</code> , <code>walking_profile</code>	texto	Ver chaves comuns.
<code>method_code</code> , <code>method_name</code>	texto	Código e nome do método.
<code>status</code>	texto	<code>ok</code> , <code>error</code> , <code>placeholder</code> , etc.
<code>run_dir</code>	texto	Caminho da pasta detalhada desta execução.

Atributos — configuração.

Atributo	Tipo	Descrição
<code>budget</code>	inteiro	Número B de POIs alocados.
<code>n_seeds</code>	inteiro	Número de sementes (reinícios multi-start).
<code>best_seed</code>	inteiro	Semente que produziu a melhor solução.
<code>objective_metric</code>	texto	Nome da métrica (<code>sum_iqc</code>).
<code>optimization_direction</code>	texto	Direção (<code>maximize</code>).
<code>rcl_alpha</code>	real	Parâmetro α da RCL.
<code>construction_sample_size</code>	inteiro	Pares avaliados por passo (<i>sampled greedy</i>).
<code>local_search_max_evals</code>	inteiro	Teto de avaliações da busca local.
<code>h3_resolution</code>	inteiro	Resolução H3 da malha.
<code>distance</code>	inteiro	Distância (m) usada na geração dos dados.

Atributos — objetivo e ganho.

Atributo	Tipo	Descrição
<code>baseline_objective</code>	real	$f(\mathbf{0})$: soma do IQC sem intervenção.
<code>best_objective</code>	real	$f(X^*)$: soma do IQC da melhor solução.
<code>improvement</code>	real	$f(X^*) - f(\mathbf{0})$ (ganho absoluto).
<code>improvement_pct</code>	real	Ganho relativo ($100 \cdot \text{improvement}/\text{baseline}$).

Atributos — distribuição entre sementes. Resumo do objetivo *após busca local* sobre as `n_seeds` execuções (útil para caracterizar a variabilidade estocástica do GRASP).

Atributo	Tipo	Descrição
<code>obj_seed_mean</code>	real	Média do objetivo entre as sementes.
<code>obj_seed_std</code>	real	Desvio-padrão amostral (estabilidade do método).
<code>obj_seed_min</code>	real	Pior objetivo entre as sementes.
<code>obj_seed_max</code>	real	Melhor objetivo (= <code>best_objective</code>).

Atributos — distribuição do IQC por hexágono. Estatísticas descritivas do IQC *entre hexágonos* (não entre sementes). Respondem diretamente à pergunta “como o IQC varia nesta localidade/perfil”.

Atributo	Tipo	Descrição
<code>iqc_base_mean, iqc_base_std</code>	real	Média e desvio do IQC <i>baseline</i> entre hexágonos.
<code>iqc_opt_mean, iqc_opt_std</code>	real	Média e desvio do IQC <i>otimizado</i> .
<code>iqc_opt_min, iqc_opt_median, iqc_opt_max</code>	real	Mínimo, mediana e máximo do IQC otimizado.
<code>delta_iqc_mean, delta_iqc_std</code>	real	Média e desvio da variação ΔIQC por hexágono.
<code>delta_iqc_min, delta_iqc_max</code>	real	Maior queda e maior aumento de IQC entre hexágonos.
<code>delta_iqc_positive_count</code>	inteiro	Quantos hexágonos tiveram o IQC <i>aumentado</i> .

Atributos — instância, solução e tempo.

Atributo	Tipo	Descrição
<code>n_hexagons</code>	inteiro	Total de hexágonos da malha.
<code>n_source_hexagons</code>	inteiro	Hexágonos-fonte elegíveis (que propagam impacto).
<code>n_candidate_dimensions</code>	inteiro	Dimensões de POI alocáveis.
<code>allocated_pois</code>	inteiro	POIs alocados na melhor solução ($= B$).
<code>distinct_pairs</code>	inteiro	Pares (hex, dimensão) distintos usados.
<code>runtime_seconds_total</code>	real	Tempo total da execução (todas as sementes).

6.4 `all_per_seed.csv` / `per_seed.csv`

O que representa. Uma linha por semente. Expõe a *variabilidade estocástica* interna: como cada reinício se saiu na construção e após a busca local.

Como usar e quais métodos aplicar.

- *Variabilidade:* média, desvio, coeficiente de variação e intervalos de confiança **bootstrap** de `local_search_objective`.
- *Contribuição da busca local:* testar se `local_search_gain` difere de zero (teste *t* pareado ou **Wilcoxon** de postos sinalizados, comparando construção vs. pós-busca-local).
- *Eficiência:* relacionar `runtime_seconds` com a qualidade (análise tempo×qualidade), base para *time-to-target*.

Atributo	Tipo	Descrição
<code>seed</code>	inteiro	Semente do gerador pseudoaleatório.
<code>construction_objective</code>	real	Objetivo ao fim da fase de construção.
<code>local_search_objective</code>	real	Objetivo após a busca local (qualidade final desta semente).
<code>local_search_gain</code>	real	Ganho da busca local (<code>local_search</code> – <code>construction</code>).
<code>improvement_over_baseline</code>	real	<code>local_search_objective</code> – <code>baseline</code> .
<code>runtime_seconds</code>	real	Tempo desta semente (construção + busca local).
<code>distinct_pairs</code>	inteiro	Pares (hex, dimensão) distintos na solução desta semente.

6.5 `all_per_hexagon_iqc.csv` / `per_hexagon_iqc.csv`

O que representa. Uma linha por hexágono (georreferenciada), com o IQC antes e depois da intervenção. É a base para a *análise espacial* dos ganhos.

Como usar e quais métodos aplicar.

- *Distribuição:* histogramas e *boxplots* de `delta_iqc` por localidade/perfil; teste de Wilcoxon pareado entre `iqc_baseline` e `iqc_optimized`.
- *Autocorrelação espacial:* **I de Moran** (global) e **LISA** (local) sobre `delta_iqc` usando `latitude/longitude` para a matriz de vizinhança — verifica se os ganhos formam aglomerados espaciais.

- *Pontos quentes*: **Getis-Ord G_i^*** para identificar *hot/cold spots* de melhoria.
- *Validação*: comparar `iqc_baseline` (recalcado pela função objetivo) com `iqc_pipeline` (valor original do pipeline) como checagem de consistência.

Atributo	Tipo	Descrição
<code>h3_id</code>	texto	Identificador H3 do hexágono.
<code>latitude, longitude</code>	real	Coordenadas do centro do hexágono.
<code>iqc_baseline</code>	real	IQC do hexágono <i>sem</i> intervenção (métrica da função objetivo).
<code>iqc_optimized</code>	real	IQC do hexágono na melhor solução.
<code>delta_iqc</code>	real	Variação <code>iqc_optimized</code> – <code>iqc_baseline</code> .
<code>iqc_pipeline</code>	real	IQC original calculado pelo pipeline (referência).

6.6 `per_hexagon_indicators.csv`

O que representa. Formato *longo* (*tidy*): uma linha por hexágono × indicador, com o valor antes/depois. Permite ver *quais indicadores* (saúde, transporte, etc.) moveram o IQC e *onde*.

Como usar e quais métodos aplicar. Decompor o ganho por dimensão; *heatmaps* indicador×hexágono; correlação entre `delta` dos indicadores e `delta_iqc`; análise de contribuição (qual indicador explica melhor a variação do IQC).

Atributo	Tipo	Descrição
<code>h3_id</code>	texto	Hexágono.
<code>indicator</code>	texto	Nome do indicador (ex. <code>S_saude</code>).
<code>value_baseline</code>	real	Valor do indicador antes da intervenção.
<code>value_optimized</code>	real	Valor após a propagação dos POIs alocados.
<code>delta</code>	real	<code>value_optimized</code> – <code>value_baseline</code> .

6.7 `all_allocations.csv` / `allocation.csv`

O que representa. Uma linha por item alocado: *onde* e *quanto* de cada dimensão foi inserido na melhor solução.

Como usar e quais métodos aplicar. Frequência de alocação por dimensão e por hexágono; mapas de calor da alocação; relação entre alocação e `delta_iqc` (cruzando por `h3_id`); padrões de alocação entre perfis (ex. idosos vs. atletas priorizam dimensões diferentes?).

Atributo	Tipo	Descrição
<code>h3_id</code>	texto	Hexágono-fonte que recebeu os POIs.
<code>dimension</code>	texto	Dimensão do POI alocado.
<code>quantity</code>	inteiro	Quantidade de POIs alocados naquele par.
<code>latitude, longitude</code>	real	Coordenadas do hexágono-fonte.

6.8 critic_weights.csv

O que representa. Os pesos CRITIC de cada indicador, *antes e depois* da intervenção. Como o CRITIC é orientado a dados, a alocação altera a importância relativa dos indicadores.

Como usar e quais métodos aplicar. Comparar `weight_baseline` vs. `weight_optimized` (gráfico de barras pareado); análise de sensibilidade do IQC aos pesos; discutir como a intervenção reequilibra a importância das dimensões.

Atributo	Tipo	Descrição
<code>indicator</code>	texto	Nome do indicador.
<code>weight_baseline</code>	real	Peso CRITIC no cenário baseline (soma = 1).
<code>weight_optimized</code>	real	Peso CRITIC na melhor solução (soma = 1).

6.9 convergence.csv

O que representa. O objetivo registrado *a cada passo* da fase de construção, para *cada* semente (formato longo). Suporta curvas de convergência e gráficos *time-to-target* (TTT), padrão na literatura do GRASP.

Como usar e quais métodos aplicar. Curvas objetivo×passo com banda de confiança (média $\pm$ IC entre sementes); *área sob a curva* como medida de rapidez de convergência; **TTT plots** (probabilidade acumulada de atingir um alvo) com ajuste de distribuição **exponencial deslocada** (Aiex, Resende e Ribeiro).

Atributo	Tipo	Descrição
<code>seed</code>	inteiro	Semente correspondente à trajetória.
<code>step</code>	inteiro	Passo de construção (1, ..., B).
<code>objective</code>	real	Valor do objetivo após inserir o k -ésimo POI.

6.10 run_metadata.json

O que representa. Os metadados completos da execução, em JSON, garantindo **reprodutibilidade** (requisito para periódicos A1/A2). Reúne, num só arquivo: identificação, configuração, objetivo, `parameters` (todos os parâmetros do método), `instance` (tamanho da instância), `seeds` (lista completa) e `environment` — versões de Python, `numpy`, `pandas`, plataforma e o *commit* do Git. Com ele, qualquer execução pode ser refeita exatamente.

6.11 run_report.txt

O que representa. Um resumo legível por humanos da execução (mesma informação do `run_summary.csv`, formatada). Serve para conferência rápida e para anexos do relatório/artigo.

6.12 Receituário de análises

A Tabela 5 mapeia perguntas de pesquisa típicas aos arquivos e métodos correspondentes.

Tabela 5: Da pergunta de pesquisa ao arquivo e ao método estatístico.

Pergunta	Arquivo(s)	Método
Um método supera os outros?	<code>all_runs_summary</code>	Friedman + pós-teste de Nemenyi (pareado por instância); ou ANOVA + Tukey.
O ganho varia por perfil/localidade?	<code>all_runs_summary</code>	ANOVA fatorial / modelo misto; η^2 como efeito.
Quão estável é o método?	<code>all_per_seed</code>	Coeficiente de variação; IC bootstrap; Wilcoxon (construção vs. busca local).
Os ganhos de IQC formam aglomerados no espaço?	<code>all_per_hexagon_iqc</code>	I de Moran (global), LISA (local), Getis-Ord G_i^* .
Quais indicadores explicam o ganho?	<code>per_hexagon_indicators</code>	Correlação <code>delta↔delta_iqc</code> ; heatmaps; contribuição.
Onde os POIs são alocados?	<code>all_allocations</code>	Frequências; mapas de calor; comparação entre perfis.
A intervenção reequilibra os pesos?	<code>critic_weights</code>	Comparação pareada baseline vs. otimizado; sensibilidade.
Quão rápido o método converge?	<code>convergence</code>	Curvas média $\pm$ IC; TTT plots (exponencial deslocada).

Ponto de partida prático. Carregar `all_runs_summary.csv` e cruzar com os demais mestres pela coluna `run_id` cobre a maior parte das análises. Os arquivos por execução entram quando se deseja o detalhe fino (curvas de convergência, mapas espaciais, decomposição por indicador).

7 Considerações finais e extensões

A implementação cobre o GRASP *tradicional* de duas fases. A estrutura modular (avaliação isolada em `_evaluate`; construção, busca local e multi-start independentes) facilita evoluções descritas por Resende e Ribeiro:

- **Reactive GRASP:** auto-ajuste do parâmetro α ao longo das iterações, em vez de fixo, aumentando a robustez;
- **Path-Relinking:** intensificação que explora trajetórias entre a solução corrente e um conjunto de soluções de elite (corresponde ao método híbrido da opção E);
- **Estratégias de reinício** (*restart*) para reduzir a cauda da distribuição de tempo até alvo.

O principal desafio prático do problema — a não-separabilidade da função objetivo (CRITIC global) — foi tratado com *sampled greedy* na construção e um orçamento de avaliações na busca local, mantendo o tempo de execução previsível sem comprometer a qualidade.

Referências

- [1] T. A. Feo and M. G. C. Resende. *Greedy randomized adaptive search procedures*. Journal of Global Optimization, 6:109–133, 1995.
- [2] M. G. C. Resende and C. C. Ribeiro. *Optimization by GRASP: Greedy Randomized Adaptive Search Procedures*. Springer, 2016.
- [3] M. G. C. Resende and C. C. Ribeiro. *Greedy Randomized Adaptive Search Procedures: Advances and Extensions*. In: Handbook of Metaheuristics, cap. 6, Springer, 2019.